UTM

UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3133 HIGH PERFORMANCE DATA PROCESSING

SECTION 01

PROJECT 1:

Optimizing High-Performance Data Processing for Large-Scale Web Crawlers

LECTURER: DR. MOHD SHAHIZAN BIN OTHMAN

Group Name: Fast&Furious

Student Name	Matric No.
GOE JIE YING	A23CS0224
NAWWARAH AUNI BINTI NAZRUDIN	A23CS0143
YASMIN BATRISYIA BINTI ZAHIRUDDIN	A23CS0201

Submission Date: 21 May 2026

Table of Contents

1.0 Introduction

The internet has made it easier for people to access large amounts of data online, but collecting and processing that data in an efficient way is still a challenge. This project focuses on using a web crawler to automatically collect data from Mudah.my. The collected data is then processed and analyzed to evaluate the performance of different data processing tools.

1.1 Project Background

Mudah.my is one of the most popular online marketplaces in Malaysia where people can buy, sell, and rent many types of things such as vehicles, electronics, jobs, and properties. For this project, the focus is on the property listings. The website has thousands of property listings from all over Malaysia. Each listing contains useful information like price, location, size, and seller details. Because there are so many listings spread across hundreds of pages, collecting this data by hand would take too long and is not practical.

To solve this, a web crawler was built to automatically collect property listing data from Mudah.my across all states in Malaysia. The data collected was then used to test and compare three data processing tools which are Pandas, Polars, and DuckDB. The aim was to find out which tool is faster and uses less memory when processing a large amount of data.

1.2 Objectives

The objectives of this project are:

1. To build a web crawler that can automatically collect property listing data from Mudah.my across all states in Malaysia.
2. To collect at least 100,000 records with 16 details per listing
3. To clean and organize the collected data so it is ready to be used for performance benchmarking and analysis.
4. To measure and record the processing time, CPU usage, memory usage, and throughput for each tool tested which are Pandas, Polars, and DuckDB.
5. To compare the results and find out which tool performs the best when processing large amounts of data.

1.3 Target Website & Data Description

The target website for this project is Mudah.my which is one of the largest online marketplaces in Malaysia. Before choosing Mudah.my, a few other websites were considered but they were either blocking the crawler as it was detected as a bot or protected by a security system. Mudah.my was chosen because it is free to access without needing to log in, has a large number of property listings across all states in Malaysia, and each listing contains enough information to build a detailed dataset.

Mudah.my organizes its listings in pages with around 43 listings per page. The URL changes using an offset value to move between pages. Each listing on the search results page only shows basic information, so the crawler needs to open each listing's own page separately to get the full details. This means the crawler works in two steps, first collecting listing links from the search pages. Then, it visits each link to get the complete information.

On the search results page, each listing displays a title, photo, price, and a summary of bedrooms, bathrooms, and property size as shown in Figure 4.1. When a listing page is opened, more details appear such as a description from the seller, land title, tenure, estimated monthly mortgage, and the seller's name as shown in Figure 4.2.

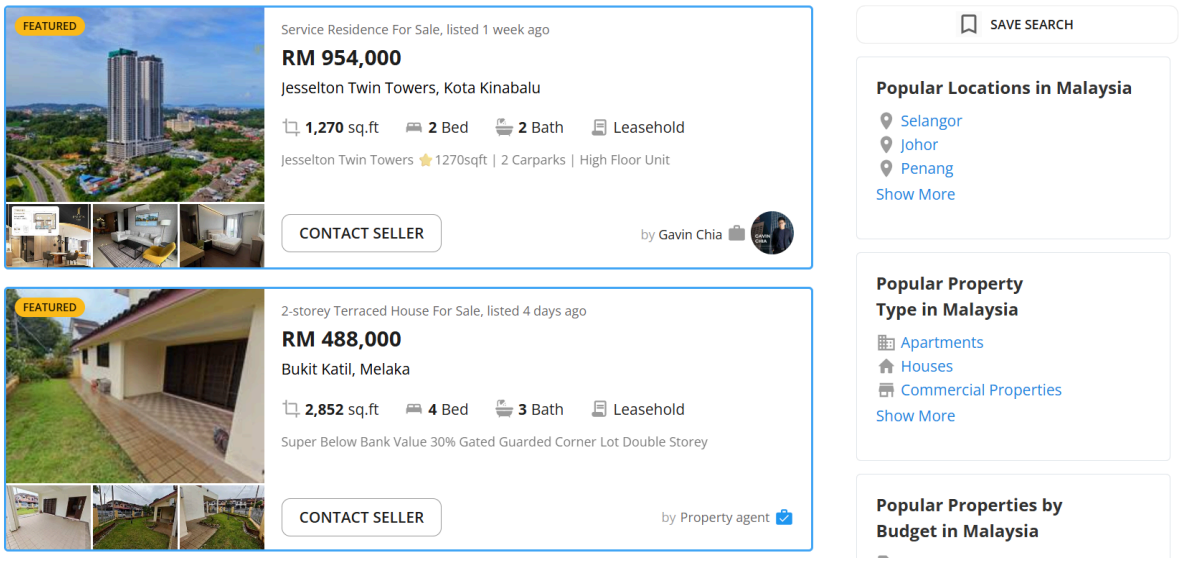

Figure 4.1 Mudah.my property search page

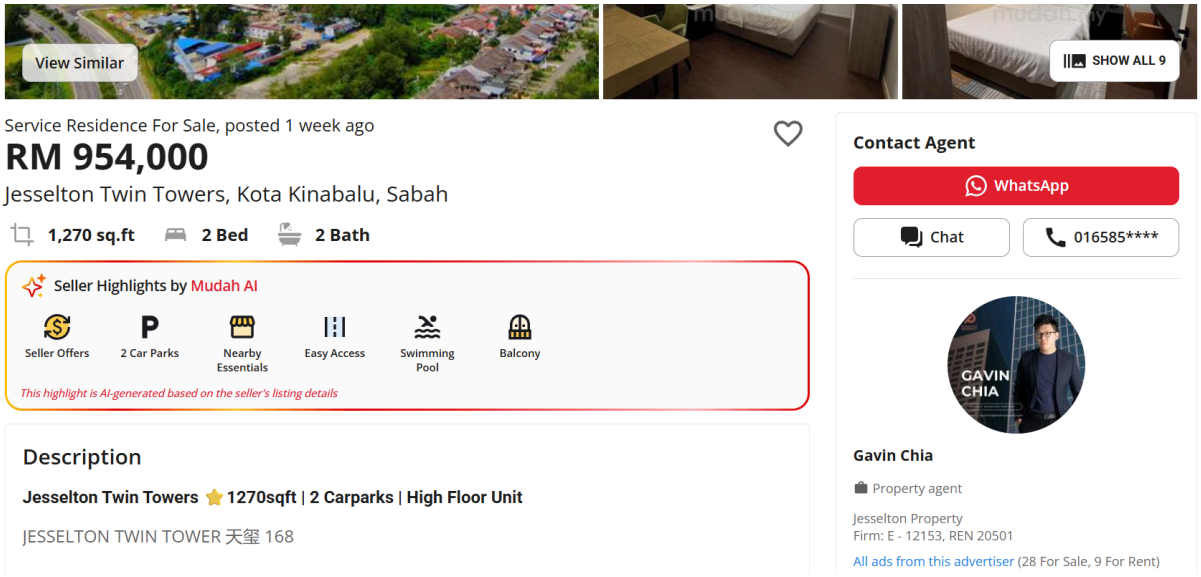

Figure 4.2 Property Details page

1.4 Extracted Data Overview

This section explains an overview of the dataset that is extracted from Mudah.my, Malaysia largest online marketplace. The data was collected across two listing categories which are property for sale and property for rent. Then, the data is merged, validated, and cleaned into a single structured dataset ready for analysis.

1.4.1. Final dataset columns

The cleaned dataset contains the same as the raw dataset which has 17 columns in total. Each column has a distinct attribute of a property listing, use of unique listing id to physical characteristics and seller information. The final column that are used are listing_id, property_type, description, location, state, price_rm, mortgage_est_rm, land_title, size_sqft, bedrooms, bathrooms, tenure, seller_type, seller_name, url, img_url, and listing type.

1.4.2 Final dataset size

The raw dataset was created by combining two separately crawled CSV files which are data_sale.csv and data_rent.csv. After merging these two files, a listing_type column was added to differentiate the source of each record. Then, the combined raw dataset contains 102,468 rows across 17 columns. By following the data cleaning steps, 2,044 duplicate rows were removed,

and then a final cleaned dataset of 100,442 records was generated. The column still remains the same as before cleaning.

1.4.3 Data type description

Mainly, the dataset contains two primary data types. Numeric columns are stored as float64 to fulfill the decimal precision and handle missing values. Then, all remaining columns are of object(string) type as they contain such as categorical values and free text after scraping from listing pages.

1.4 Basic structure after crawling

After initial extraction, the raw dataset reflected the unprocessed state of scrapped web data, which usually contains inconsistencies such as mixed data types, HTML remnant duplicate records, and missing values. The cleaning pipeline has transformed the raw data into a more structured, and analysis ready form. Finally, the cleaned dataset is the final version of data that is used for all subsequent analysis in this report.

1.5 Team Roles

In completing this project, the task has been distributed equally among the team members. The task has been distributed as shown in Table 1.5

Member name	Task
Nawwarah Auni Binti Nazrudin	Focus on Pandas optimization, including performance measurement of total processing time, CPU usage, memory utilization, and throughput. Moreover, incharge of main crawler documentation via main_crawler.ipynb, where added a step by step processing overviews and comprehensive code documentation.
Yasmin Batrisyia Binti Zahiruddin	Take over the polars optimization implementation, including performance measurement of total processing time, CPU usage, memory utilization, and throughput. Furthermore, in charge of

	developing the central data cleaning pipeline to recheck the raw crawled datasets (raw_data.csv) into a finalized standardized output (cleaned_data.csv)
Goe Jie Ying	In charge of DuckDB optimization implementation, including performance measurement of total processing time, CPU usage, memory utilization, and throughput. Then, integrate the individual results into an automated pipeline (optimize_pipeline.ipynb) with CSV exports. Next, in charge of developing the visual analysis framework (evaluation_charts.ipynb)

Table 1.5 Team roles

2.0 System Design and Architecture

This section describes the overall system architecture developed for the project. It shows the diagrams to clearly explain the workflow of the web crawling pipeline, the tools and frameworks used and the data flow in the project.

2.1 System Architecture

The overall workflow of the system is organized into several layers, beginning from data collection and ending with performance evaluation and visualization.

1. **Network Layer -** The entry point of the system. Uses the internet and HTTP requests to reach external websites.
2. **External Data Sources -** Targets Mudah.my as the data source for property listings.
3. **Application Layer -** The core processing engine, divided into three modules:
    - Web Scraper Module: Sends HTTP requests to Mudah.my, extracts listing data, and parses HTML using BeautifulSoup, saving the output as CSV.
    - Data Cleaning Module: Removes duplicates, fixes missing values, and standardizes formatting to prepare the data for analysis.
    - Optimization Module: Processes the cleaned data using Pandas, Polars, and DuckDB to benchmark and compare their performance.
4. **Storage Layer** - Two Excel files act as checkpoints:
    - Raw Scraped Dataset – output of the scraping process
    - Cleaned Dataset – output of the data cleaning process
5. **Presentation Layer** - Final output is a Performance Metric Chart Comparison, visualizing how Pandas, Polars, and DuckDB compare when processing the same dataset.

The system architecture diagram is as shown in Figure 2.1.

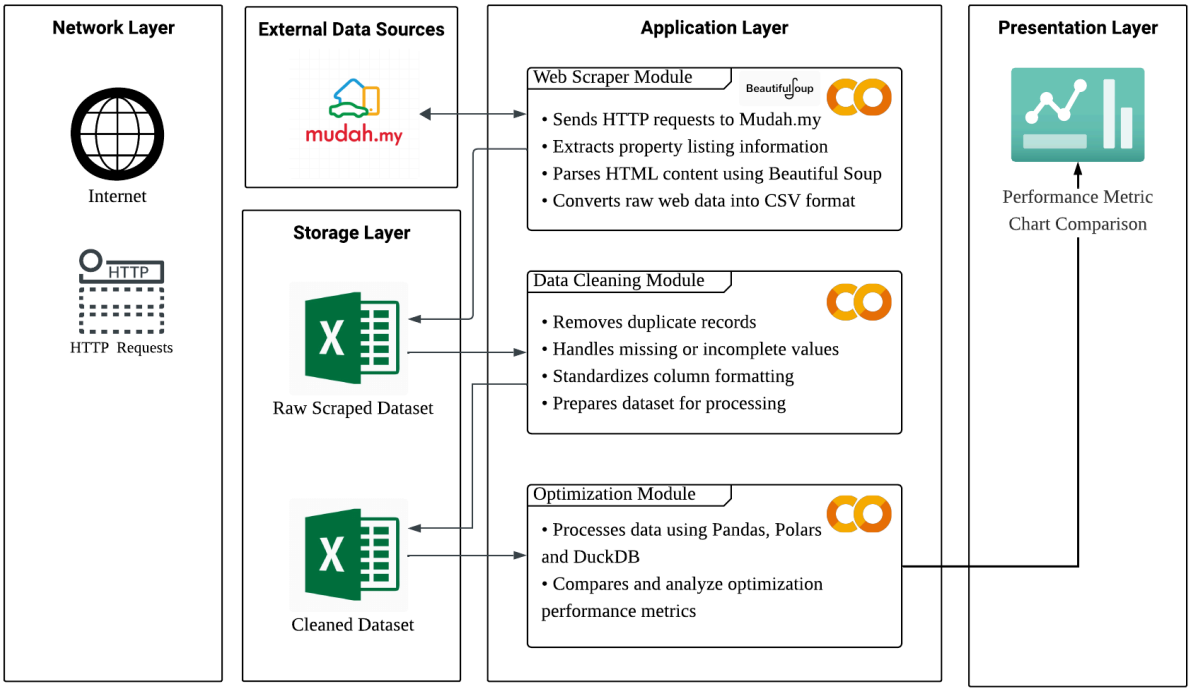

Figure 2.1 System Architecture diagram

2.2 Architecture of Tools and Frameworks Used

The architecture diagram of tools and framework used is as shown in Figure 2.2 and workflow explanation as below:

1. Google Colab runs BeautifulSoup to scrape data from mudah.my, producing raw data.
2. The raw data is passed into Pandas for cleaning and transformation, producing clean data.
3. Three tools process the clean data in parallel as shown in Table 2.2

Pipeline	**Tool**	**Status**	**Output File**
Baseline	Pandas	Before optimization	performance_before.csv
Optimized	Pandas	After optimization	performance_after.csv
Optimized	Polars	After optimization	performance_after.csv
Optimized	DuckDB	After optimization	performance_after.csv

Table 2.2 process the clean data in parallel

All four CSV outputs are saved into a central database.

4. The database flows into data metrics analysis, then into data visualization to show charts and graphs for insights.
5. The clean data and final outputs are pushed to GitHub for version control and sharing.

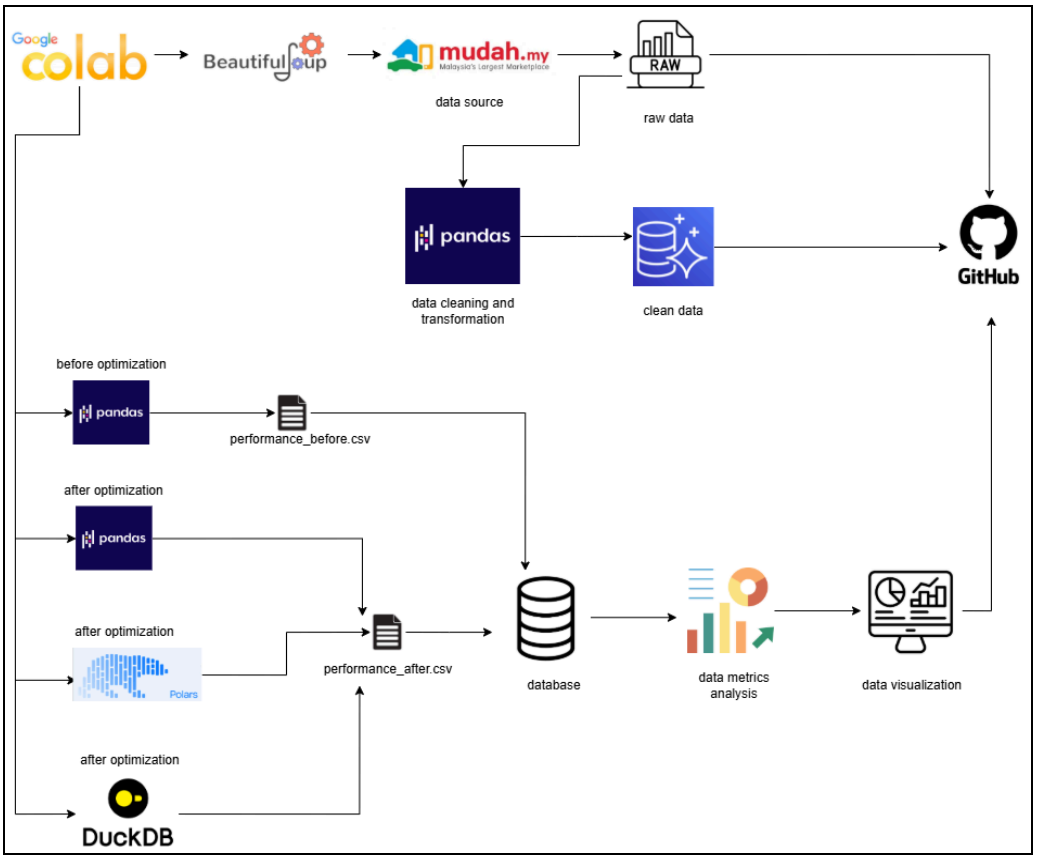

Figure 2.2 Architecture diagram of tools and framework used

3.0 Data Collection

This section explains how the data was collected from Mudah.my. It covers the method used to crawl the website, the size and structure of the raw dataset, the contribution of each team member during the crawling process, and the ethical considerations taken into account throughout the data collection phase.

3.1 Crawling Methodology

The web crawler was built using Python and was run on Google Colab. This project uses two libraries. The web crawler used the requests library to send HTTP requests to the website. It also used the BeautifulSoup library to read and extract information from the HTML code of the website. The web crawler went through each states listing pages one by one. It used a value in the URL to move between pages.

For every listing the web crawler found it first collected information from the search results page. Then it opened each listings page to get the full details of the listing. To avoid getting blocked by the website, the requests were sent with browser-like headers. If a page failed to load the web crawler would try again up to three times before moving on to the page. All the data the web crawler collected was saved into CSV files. These CSV files were uploaded to a shared Google Drive folder. The full source code for the web crawler is available on the team's GitHub repository.

3.2 Data Size and Structure

The raw dataset was collected from Mudah.my and combined into a single file. It contains 16 columns before any cleaning was done. The table below shows the structure of the raw dataset before cleaning. The structure of the raw dataset is shown in Table 3.1.

Table 3.1 Data Description

No.	Column	Description
1	listing_id	Unique ID for each listing
2	property_type	Type of property

3	description	Description written by seller
4	location	Area of the property
5	state	State in Malaysia
6	price_rm	Price in Ringgit Malaysia
7	mortgage_est_rm	Estimated monthly mortgage
8	land_title	Land title type
9	size_sqft	Size of the property in square feet
10	bedrooms	Number of bedrooms
11	bathrooms	Number of bathrooms
12	tenure	Freehold or leasehold
13	seller_type	Agent or individual seller
14	seller_name	Name of the seller
15	url	Link to the listing page
16	img_url	Link to the listing image

3.3 Team Contribution & Crawling Effort

During the data collection phase, the crawling work was divided among the three team members so that the process could be finished faster. Each member was given a different set of states to scrape and used the same crawler notebook on their own Google Colab. The contribution of each member is shown in Table 3.2.

Table 3.2 Team Contribution for Web Scraping

Member Name	Task (Assigned States)	Time Spent (Minutes)	Records Collected
Goe Jie Ying	Johor	753	10806
	Kedah	562	8702
	Kelantan	105	1671

	Melaka	597	8127
	Negeri Sembilan	686	10043
Nawwarah Auni	Selangor	720	10419
	Kuala Lumpur	708	10367
	Perak	259	6538
	Pahang	285	3998
	Labuan	11	71
Yasmin Batrisyia	Sabah	550	8046
	Sarawak	720	10607
	Perlis	25	334
	Putrajaya	88	1263
	Terengganu	42	549
	Kuala Lumpur	698	10201
	Penang	670	9883
		Total Records Collected	111,625

3.4 Ethical Considerations

The data collection process was carried out responsibly and ethically. The crawler only accessed pages that are publicly available on Mudah.my. No login was required and no private information was accessed. The crawler was also designed to send requests at a reasonable pace to avoid putting too much load on the website's server. The data collected only includes information that sellers have chosen to share publicly on their listings. All data collected in this project is strictly used for academic purposes as part of a university assignment and is not used for any commercial activity.

4.0 Data Processing

The raw dataset was collected from four separate CSV files that were scrapped from Mudah.my by different team members. Before beginning the analysis, the data required to clean and standardize to resolve data quality issues such as inconsistencies, remove duplicates, handle missing values, and ensure correct data types. All this process was carried out in Python using pandas in Google Colab, and the final cleaned dataset was exported as a single CSV file for further analysis.

4.1 Data Cleaning Process

By merging the four separate CSV files (mudah_auni.csv, mudah_yasmin.csv, mudah_goe.csv, and data_rent.csv) using pandas concat function, the raw dataset is created with **111,620 rows and 16 columns.** When merging the raw dataset solely, the data structure is inconsistent across the file. For example, the seller_name column was not included in some sources and was added with null values to ensure schema is aligned.

Once combined, several cleaning operations were applied:

4.1.1 Duplicate removal

This process is performed by using drop_duplicates() to eliminate redundant records that appear from overlapping data collection across team members.

4.2.2 Missing value treatment

This process is done based on column-type strategy where text and categorical columns were filled with the placeholder ‘N/A’ to preserve row completeness, while numerical columns were placed by their respective median values to minimise the effect of outliers. Rows with all null values were entirely dropped outright.

4.2.3 HTML tag stripping

This process applied to all non-URL text columns by using a regex based function(re.sub(r'<[^>]+>', '', ...)), as the data was scrapped from a web listing platform and usually contained embedded HTML markup in fields such as description and location.

4.2.4 URL validation

This is applied to all URL-type columns by flagging and nullifying entries that did not start with a valid path such as http:// or https:// prefix, rather than dropping the associated rows.

4.2.5 Outlier

Outliers were applied to bedrooms and bathrooms fields, where if the value is more than 20, it is replaced with null. This is to prevent data entry errors inconsistent with real residential listings.

4.2 Data Transformation Process

By following the cleaning stage, several transformation steps were carried out to standardise all the dataset for the analysis.

4.2.1 Column name normalisation

This process was applied through all columns such as names were lowercase, stripped of leading/trailing whitespace, spaces were replaced with underscore, and non-alphanumeric characters were removed. This is to ensure consistent and programmatically accessible column names.

4.2.2 Whitespace normalisation

This applied to all string columns, collapsing multiple consecutive spaces into a single space and trimming edge whitespace.

4.2.3 Numeric type enforcement

This price_rm, size_sqft, mortgage_est_rm, bedrooms, and bathrooms columns were all coerced to numeric type using pd.to_numeric(..., errors='coerce'), with non-parseable values converted to null. Price related string columns had non-digit characters removed due to conversion.

4.2.4 Datetime conversion

This applied to any column that contains date, time, or crawl related terms using pd.to_datetime(..., errors='coerce') to produce proper type temporal fields.

4.2.5 Listing ID formatting

Listing ID was standardized by converting to strings, stripping whitespace, and removing floating-point suffixes that appeared during file loading.

4.3 Data Structure

The cleaned dataset has 100,442 rows and 16 columns after processing and is saved in a cleaned_data file . The structure of the dataset is as follows:

No.	Column	Description
1	listing_id	Unique ID for each listing
2	property_type	Type of property
3	description	Description written by seller
4	location	Area of the property
5	state	State in Malaysia
6	price_rm	Price in Ringgit Malaysia
7	mortgage_est_rm	Estimated monthly mortgage
8	land_title	Land title type
9	size_sqft	Size of the property in square feet
10	bedrooms	Number of bedrooms
11	bathrooms	Number of bathrooms
12	tenure	Freehold or leasehold
13	seller_type	Agent or individual seller
14	seller_name	Name of the seller
15	url	Link to the listing page
16	img_url	Link to the listing image

5.0 Optimization Techniques

This section presents the optimization techniques applied to improve the performance of data processing pipeline. Three approaches implemented include Pandas Optimized Pipeline, Polars and DuckDB.

5.1 Pandas Optimized Pipeline

The Pandas optimized pipeline reduces unnecessary memory usage and enhances computation efficiency during data loading and processing to improve performance. Several optimization strategies were applied using Pandas:

- usecols: load needed columns only
- dtype: reduce type inference overhead
- .astype('category'): reduce memory for repeated text, such as state
- .str.strip().str.title(): vectorized string processing
- mask_house: fast vectorized filtering
- groupby observed=True: skip empty categories
- pd.cut: vectorised binning

These optimizations were implemented in Pandas optimized pipeline as shown in following code snippet:

```python
df = pd.read_csv(
    CSV_FILE,
    usecols=existing_cols,
    dtype={'listing_id': 'str'},
    engine='c'
)

df['property_type'] = df['property_type'].astype('category')

df['location'] = df['location'].str.strip().str.title()
```

```
mask_house = df['property_type'].astype(str).str.contains(
    'house|bungalow|terrace|semi.d|villa|townhouse', case=False, na=False
)
df_house = df.loc[mask_house]
```

5.2 Polars

The Polars pipeline is designed to improve performance through lazy evaluation and parallel execution. Polars optimizes queries before execution to allow faster computation and better resource utilization. The optimization strategies that applied using Polar were:

- scan_csv: lazy loading to avoid immediate computation
- schema casting: define data types early to reduce overhead
- .filter(): efficient vectorized filtering
- .group_by().agg(): parallel aggregation operations
- expression API: optimized column-based computation
- lazy execution (collect() at final stage)

These optimizations were implemented in Polars pipeline as shown in following code snippet:

Python
```
lf = pl.scan_csv(CSV_FILE)
lf_houses = lf.filter(
    pl.col('property_type').str.contains(
        'house|bungalow|terrace|semi.d|villa|townhouse'
    )
)

state_summary = (
    lf_houses
    .group_by('state')
    .agg([
        pl.mean('price_rm').alias('mean_price'),
        pl.median('price_rm').alias('median_price'),
```

```
        pl.count()
    ])
)
df_state = state_summary.collect()
df_houses = lf_houses.collect()
```

5.3 DuckDB

The DuckDB pipeline uses an in-process analytical SQL engine to perform high-performance data processing on CSV files. By using DuckDB to execute optimized SQL queries, there is no need to load data fully into memory, thereby suitable to process large datasets and aggregation-heavy workloads efficiently. The optimization methods applied DuckDB were:

- read_csv_auto(): direct query execution on CSV without full loading
- TRY_CAST: safe type conversion during query execution
- WHERE filtering: early data reduction at query level
- REGEXP_MATCHES: efficient pattern-based filtering
- SQL aggregation (AVG, MEDIAN, COUNT)
- GROUP BY optimization for analytical processing

These optimizations were implemented in DuckDB pipeline as shown in following code snippet:.

```
Python
SELECT
    state,
    AVG(TRY_CAST(price_rm AS DOUBLE)) AS mean_price,
    MEDIAN(TRY_CAST(price_rm AS DOUBLE)) AS median_price,
    COUNT(*) AS count
FROM read_csv_auto('data.csv')
WHERE                                                REGEXP_MATCHES(property_type,
'house|bungalow|terrace|semi.d|villa|townhouse', 'i')
GROUP BY state
ORDER BY mean_price DESC;
```

6.0 Performance Evaluation

This section evaluates the performance of different data processing methods used in the project by conducting comparison between optimization techniques. The unoptimized Pandas implementation served as the baseline benchmark, while Pandas Optimized pipeline, Polars and DuckDB were used as the optimization approaches to be evaluated and compared.

To ensure fair comparison, all methods were executed using the same laptop and same cleaned dataset. Each method was evaluated using several performance metrics, including total processing time, CPU usage, memory usage and throughput.

6.1 Performance Metrics Table

The performance evaluation was conducted using four key metrics to compare the performance of the baseline and optimized processing methods. The selected metrics include:

- **Total Processing Time (seconds):** The time required to complete the processing pipeline
- **CPU Usage (%):** The processor utilization recorded during execution
- **Memory Usage (MB):** The amount of memory consumed by each processing method
- **Throughput:** The number of records processed per second

The benchmark results for these performance metrics are presented in the following tables.

Table 6.1.1 Comparison for Total Processing Time (seconds)

Optimization Stage	Run 1	Run 2	Run 3	Average
Pandas Baseline	11.20	4.19	4.62	6.67
Pandas Optimization	3.73	5.05	3.93	4.24
Polars Optimization	0.93	0.83	0.75	0.84
DuckDB Optimization	1.33	1.02	1.29	1.21

Table 6.1.2 Comparison for CPU Usage (%)

Optimization Stage	Run 1	Run 2	Run 3	Average
Pandas Baseline	47.0	91.7	97.4	78.7
Pandas Optimization	100.0	98.4	100.0	99.47
Polars Optimization	113.7	124.7	132.7	123.7
DuckDB Optimization	107.2	136.4	110.3	117.97

Table 6.1.3 Comparison for Memory Usage (MB)

Optimization Stage	Run 1	Run 2	Run 3	Average
Pandas Baseline	251.16	251.16	251.17	251.17
Pandas Optimization	213.61	213.61	213.61	213.61
Polars Optimization	0.09	0.04	0.04	0.06
DuckDB Optimization	0.15	0.15	0.15	0.15

Table 6.1.4 Comparison for Memory Usage (MB)

Optimization Stage	Run 1	Run 2	Run 3	Average
Pandas Baseline	8970	23952	21738	18220
Pandas Optimization	26930	19905	25544	24126
Polars Optimization	58641	65582	73360	65861
DuckDB Optimization	40965	53658	42470	45698

6.2 Comparison Charts

To provide a clearer visualization of the benchmark results, comparison charts for the four performance metrics, including total processing time, CPU usage, memory usage and throughput were generated and presented in Figure 6.2.1. The charts show the comparison between Pandas baseline, Pandas optimized pipeline, Polars and DuckDB, while the best-performing method for each metric is highlighted in yellow.

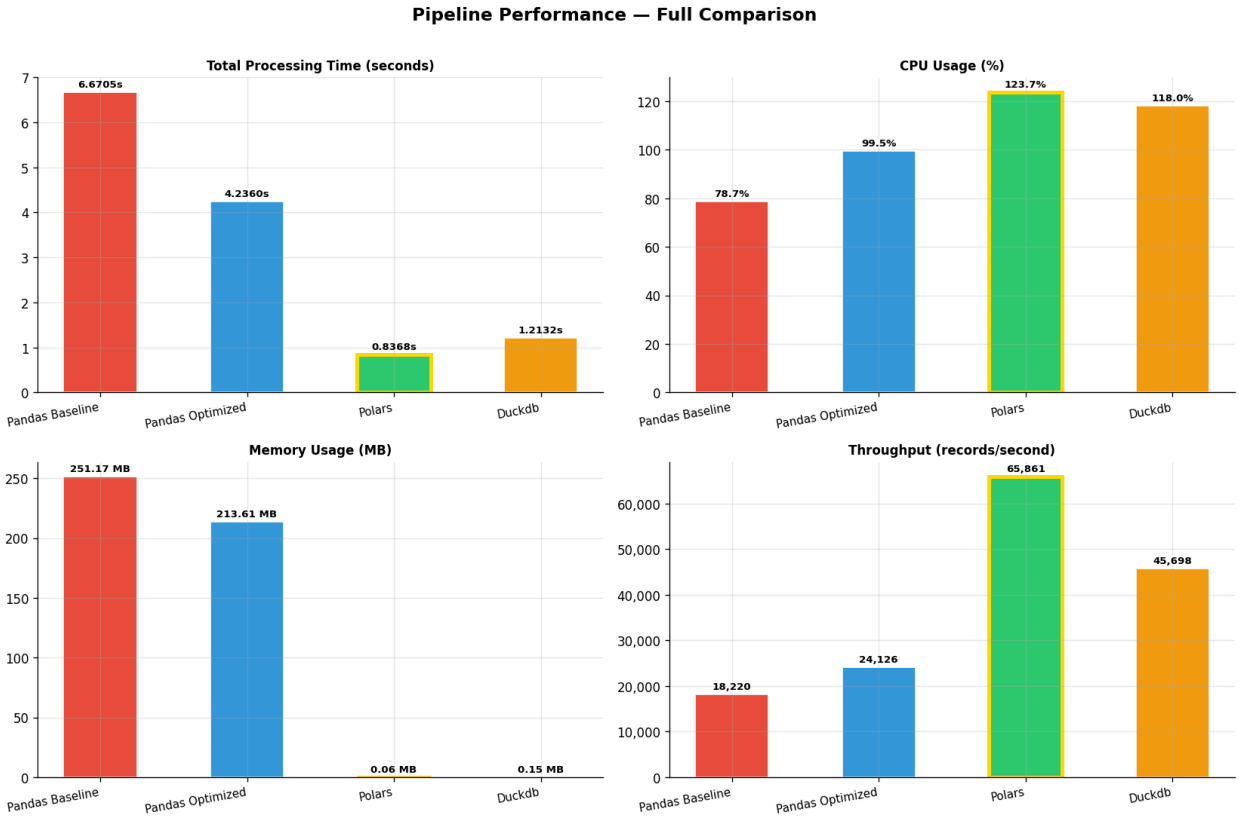

Figure 6.2.1 **Comparison Charts**

The benchmark result indicates that all the optimization techniques improved the performance of the data processing pipeline compared to the Pandas baseline. The baseline implementation recorded the highest processing time and memory usage, lowest CPU utilization percentage and throughput per second, which is inefficient especially for handling large datasets.

After optimization, the Pandas optimized pipeline showed improved execution performance compared to the baseline. Polars achieved the best overall performance among all evaluated methods, with the lowest memory usage and processing time, along with highest CPU utilization efficiency and throughput per second. DuckDB also presented competitive performance with efficient query execution and stable memory usage.

Overall, the findings indicate that optimized methods consistently outperform the baseline approach. Among the tested optimization strategies, **Polars** is the most effective for large-scale data processing, obtaining the best performance in both speed and resource utilization among these three optimization strategies.

7.0 Challenges and Limitations

During the development of this project, several challenges and limitations were encountered. One of the main challenges faced was that the initially selected websites cannot be successfully scrapped due to restrictions. In some cases, the website implemented security mechanisms to block the web crawler as it was detected and identified as automated bots. Additionally, one of the websites was protected by Cloudflare, which prevented direct data extraction. As a result, the team had revised the website selection multiple times. Finally, mudah.my was chosen as the data source for this project.

In addition, although the website displayed more than 200,000 property for sale records, access to view was limited to the first 250 pages of listings only, with approximately 43 records per page. This causes the target of at least 100,000 records to not be reached. To overcome this limitation, the crawling process was divided according to property listings by states, enabling the team to collect 250 pages for each state separately to reach the desired target.

Another challenge faced involved obtaining a dataset which was sufficiently large for optimization comparison. The initial dataset scraped was only 6 columns, the details for each record were also limited to short string and numeric value only. It is considered insufficient for evaluating and comparing different optimization methods used, such as Pandas Optimized, Polars and DuckDB. Therefore, the crawling process was redesigned and restarted in a short period of time to collect a new dataset consisting of 16 columns from properties for sale and properties for rent category to be used as the final raw dataset in this project.

Data quality issues were also presented in the raw dataset. Several columns, such as bathrooms, bedrooms and seller_name contained missing values. This occurred because not all property listings in the website were residential houses and some of them did not provide complete information about the property. Consequently, appropriate methods to handle missing values were done before benchmarking and analysis.

Furthermore, time constraints became a limitation throughout the project. The need to repeatedly revise the website to perform crawling and rescrap the original project disrupted the original project schedule. This causes delay progress to the next step, such as optimization and documentation. Despite these constraints, the team was still able to realign the task and workflow

to complete the project within the time given. This helped the team to understand real world web crawling and data project challenges and also prepared the team to adjust quickly when facing unexpected issues.

8.0 Conclusion and Future Work

8.1 Conclusion

This project has successfully demonstrated the end-to-end process of large scale web crawling and high performance data processing from the website mudah.my which is the most popular online marketplaces in Malaysia as the data sources. The web crawler was created by using Python, Requests, and BeautifulSoup to automatically collect the property listing data across all states in Malaysia and as a result, the cleaned dataset retrieved was 100,442 rows with 16 columns.

The objective of benchmarking by using three data processing tools (Pandas, Polars, and DuckDB) was achieved using four performance metrics which are processing time, CPU usage, memory usage, and throughput. The result clearly showed that all optimized approaches are better than Pandas baseline. Among the tools evaluated, Polars shows the best overall performance by achieving the lowest processing time of 0.84 second, the lowest memory consumption of approximately 0.06 MB, and the highest throughput of around 65,861 records per second. Meanwhile, DuckDB also shows a good and competitive performance particularly in memory efficiency and query execution stability.

This project shows that the choice of data processing tool also has a significant impact on performance when handling the large-scale datasets. For high-performance data processing tasks involving large CSV datasets, polars is the most suitable tool due to its lazy evaluation, multi-threaded execution, and minimal memory footprint. DuckDB also offers an alternative way for users who prefer SQL-based workflows as it enables direct querying on raw CSV files without fully loading data into memory.

Overall, this project has provided valuable practical experience in web crawling, data cleaning, and performance benchmarking, while also highlighting real challenges such as bot detection, website access restriction, and data quality issues solved during the development process.

8.2 Future work

While this project has achieved its objectives, some areas can be improved and extended in future work:

1. Incorporating Real-Time crawling

   Implement a scheduled or incremental crawling pipeline that periodically updates the dataset to track price changes, new listings and removed listings over time.

2. Multimodal data integration

   As the current dataset consists mostly of structured text and numeric fields. Future work could incorporate unstructured data such as property listing images to build a multimodal pipeline.

3. Benchmarking additional processing engines

   Extend the benchmark tools by including other modern data processing engines such as Apache Arrow, or Vaex so that could provide a more comprehensive comparison across a wider range of tools and hardware configurations.

9.0 References

Python Data Bench. (2026, May 23). Beyond Pandas: A practical guide to polars and DuckDB for Python data science. *Python Data Bench*. https://pythondatabench.com/article/beyond-pandas-practical-guide-polars-duckdb-python-data-science

Anudeep Mahavadi, & Atchutanna Subodh. (2026, March 22). Pandas vs Polars vs DuckDB: What Data Scientists Should Use in 2026. *Analytics Insight: Top Tech & Crypto Publication | Latest AI, Tech, Crypto News*. https://www.analyticsinsight.net/programming/pandas-vs-polars-vs-duckdb-what-data-scientists-should-use-in-2026

GeeksforGeeks. (2025, July 23). *Best data cleaning techniques for preparing your data*. GeeksforGeeks. https://www.geeksforgeeks.org/data-science/best-data-cleaning-techniques-for-preparing-your-data/

Mudah.my | Malaysia's largest marketplace | Buy & sell new and used items online. (n.d.). Mudah.my - Malaysia's Largest Marketplace. https://www.mudah.my/

10.0 Appendices

Links to all the repository of the code and dataset

10.1 Links to full code repo or dataset

Group Github: HPDP/2526/project/p1/Fast&Furious at main · drshahizan/HPDP

Dataset: HPDP/2526/project/p1/Fast&Furious/p1/data at main · drshahizan/HPDP

Main crawler: HPDP/2526/project/p1/Fast&Furious/p1/main_crawler.ipynb at main · drshahizan/HPDP

Data cleaning:HPDP/2526/project/p1/Fast&Furious/p1/Data_cleaning (3).ipynb at main · drshahizan/HPDP

Optimization: HPDP/2526/project/p1/Fast&Furious/p1/optimize_pipeline.ipynb at main · drshahizan/HPDP